

QualiJournal Admin System Code Analysis and Issue Resolution

1. 파일 간 정합성 및 경로 일치 확인

현재 프론트엔드(admin/index.html)의 **fetch API 호출 경로**들과 백엔드(server_quali.py)의 **FastAPI 엔드포인트 정의**가 잘 일치합니다. 예를 들어:

- **보고서 생성:** 프론트에서 `/api/report` 로 POST 요청을 보내는 함수가 있으며 ¹, 백엔드에도 `/api/report` POST 엔드포인트가 정의되어 있습니다 ². 둘 다 `date` 와 `keyword` 를 받아 일일 보고서를 생성하고, 생성된 MD 파일 경로(`path`) 등을 JSON으로 반환합니다.
- **요약/번역 요청:** 프론트의 `enrichKeyword()` 와 `enrichSelection()` 함수는 각각 `/api/enrich/keyword` 및 `/api/enrich/selection` 엔드포인트에 POST 호출을 합니다 ³. 백엔드에도 동일한 경로의 POST 핸들러(`enrich_keyword`, `enrich_selection`)가 구현되어 있어 키워드 전체 기사와 선정본 기사에 대한 요약/번역 파일을 생성하고 경로를 반환합니다 ⁴ ⁵ ⁶.
- **Export 기능:** 프론트의 `exportMd()` 와 `exportCsv()` 는 `/api/export/md` 및 `/api/export/csv` GET 요청을 통해 Markdown 혹은 CSV 파일을 Blob으로 받아 다운로드합니다 ⁷ ⁸. 백엔드에서는 `/api/export/{fmt}` 로 포맷별 Export를 처리하고, 특별히 `preview` 쿼리 파라미터가 있으면 Content-Disposition 헤더 없이 미리보기용 콘텐츠만 반환하도록 처리합니다 ⁹. 또한 안정성을 위해 `/api/export/md` 등의 별도 alias 경로도 정의해 두었습니다 ¹⁰. 따라서 **미리보기(Preview)** 버튼 역시 `/api/export/md?preview=1` 경로로 호출되어 백엔드에서 Markdown 텍스트를 바로 받아 모달에 표시합니다 ¹¹ (백엔드도 해당 파라미터를 인지하여 미리보기 기능을 제공합니다).
- **기타 경로:** KPI 상태 조회(`/api/status`), 게이트 설정 갱신(`/api/config/gate_required`), 최근 작업 조회(`/api/tasks/recent`), 작업 취소(`/api/tasks/{id}/cancel`), Ready 목록 조회(`/api/items?state=ready`), 개별 기사 발행(`/api/items/{id}/publish`) 등 프론트에서 호출하는 경로들은 **모두 백엔드에 대응되는 엔드포인트가 있습니다**. 예를 들어 Ready 목록의 “발행” 버튼은 `/api/items/{id}/publish` 로 POST 호출하며 ¹², 백엔드에도 해당 경로로 승인/발행 처리 엔드포인트가 구현되어 있습니다 ¹³.

결론: 주요 API 경로와 HTTP 메시드가 프론트와 백엔드 간에 일치하므로, **경로 불일치로 인한 호출 오류는 발견되지 않았습니다**. 예컨대, admin UI에서 `/api/report` 를 호출하면 백엔드의 `/api/report` 핸들러가 정확히 받아 처리하고 결과(`{ "ok": true, "path": ... }` 형태)를 반환하도록 정합성이 확보되어 있습니다 ¹ ². 다만 **아주 경미한 부분**으로, 게이트 조절 PATCH 호출 시 프론트는 `{gate_required:값, value:값}` 형태로 보내는데 ¹⁴, 백엔드 모델은 `gate_required` 필드만 사용합니다 ¹⁵. 이 때 Pydantic 모델이 추가 필드를 무시하도록 설정되어 있어 충돌은 없으며, `gate_required` 값을 정상 반영(`GATE_REQUIRED`)하고 있습니다. 이는 과거/다른 구현과의 호환성을 고려해 프론트에서 `value` 도 같이 보내는 것으로 보이며, 실제 동작에는 문제가 없습니다.

2. 주요 이슈 해결 여부 (1027 보고서 지적사항 기준)

사용자가 제공한 "1027_2 QualiJournal 관리자 주요 기능 문제 분석 및 해결책" 문서에서 지적된 문제들이 최신 코드상 해결되었는지 검토한 결과, **대부분 개선/해결된 것으로 확인됩니다**. 주요 이슈별로 살펴보면:

- **“토큰 해제 안됨” 문제:** 관리자가 인증 토큰을 UI에서 제거(log out)해도 효과가 없었던 문제로 추정됩니다. 최신 코드에는 토큰 제거 기능 `clearAdminTokenJJ()` 가 구현되어 있습니다 ¹⁶. 이 함수는

localStorage에서 ADMIN_TOKEN을 삭제하고 입력 필드를 비우며, UI 상태를 즉시 갱신해 “토큰 해제 완료” 토스트 메시지를 표시합니다 17. 또한 refresh()와 testToken()을 호출하여 토큰 제거 후 상태를 바로 재조회함으로써, 토큰이 없는 상태를 UI에 반영합니다 18. 따라서 토큰 해제 기능은 정상적으로 동작하며, 이슈가 해결된 것으로 판단됩니다. 실제 UI에서도 “해제” 버튼 클릭 시 토큰이 삭제되고 상단 상태 배지에 ‘미인증’으로 변경되는 등의 동작이 확인됩니다.

- “KPI 미갱신” 문제: 관리자 대시보드의 주요 지표(KPI) 숫자들이 최신 상태로 갱신되지 않는 문제였습니다. 현재는 페이지 로드 시 자동으로 KPI를 한 차례 새로고침하고, 수동 새로고침 버튼 (id=btnRefresh와 Ready 패널의 btnKpiRefresh)도 제대로 작동합니다 19 20. 백엔드 /api/status 엔드포인트는 승인된 기사 수, 준비 완료 수 등 KPI 정보를 토큰 보호 하에 계산하여 JSON으로 반환하며 21 22, 프론트의 refresh() 함수가 이를 호출해 KPI 숫자를 갱신합니다 23 24. 특히 과거에는 /api/status가 인증 없이 열려 있어 잘못된 인증 상태 표시 버그가 있었으나, 현재는 해당 API도 토큰 검증을 거치도록 수정되어(항상 유효 토큰 필요) UI에서 정확한 인증/KPI 상태를 반영합니다 25 26. 자동 새로고침(30초 간격)도 기본 활성화되어 있어 주기적으로 KPI를 갱신하며, 토큰 없을 시에는 동작하지 않도록 예외 처리되어 있습니다 27 28. 이러한 수정으로 KPI 지표 미갱신 문제가 해결되었습니다.

- 인증 401/403 오류 및 혼선: Cloud Run의 인증 방식(특히 인증이 필요한 Cloud Run 설정 vs. 애플리케이션 토큰)으로 인해 발생한 401/403 오류가 지적되었습니다. 현행 구현에서는 Cloud Run 서비스를 --allow-unauthenticated로 배포하는 것을 전제로, 애플리케이션 레벨에서 ADMIN_TOKEN 검증을 수행합니다 29 30. 백엔드 authorize 미들웨어는 ADMIN_TOKEN 또는 API_TOKEN 환경변수가 설정된 경우에만 토큰을 요구하며, 둘 중 하나라도 맞으면 통과시킵니다 31 32. 덕분에 Cloud Run이 퍼블릭 서비스로 운영될 때도 자체 토큰으로 보호되고, 만약 Cloud Run이 비공개(인증필요) 모드로 설정되더라도 Authorization 헤더에 담기는 Cloud Run의 ID 토큰과는 별개로 X-Admin-Token을 통해 별도 앱 토큰을 받아 처리하도록 가이드하고 있습니다 33 34. 프론트엔드에서는 fetch 호출 시 Authorization 헤더와 X-Admin-Token 헤더를 모두 자동 부착하여 두 체계를 양립시켰습니다 35. 이러한 일관된 처리 덕분에, 이전에 보고된 “토큰이 존재해도 401/403 발생” 등의 문제는 개선되었습니다. 이제 유효한 토큰을 입력하면 대부분의 API 호출에서 401 오류 없이 정상 응답을 얻고, 토큰 미보유/오류 시에는 UI 토스트로 안내하고 API 호출을 막는 등 예외처리도 강화되었습니다 36 37.

그 밖에 PDF에서 언급되었을 가능성이 있는 세부 이슈들(예: KPI 0으로 초기화, 토큰 입력 UX, 승인 UI 링크 오류 등)도, 코드 상으로는 적절히 처리되었음을 확인했습니다. 예컨대 updateAuthUI()를 통해 토큰 유효 여부에 따라 상단 상태 배지 색상/텍스트를 “인증됨/미인증”으로 갱신하고 있고, 승인 UI 열기 기능도 /api/approve-ui/start API를 통해 동적 생성된 UI URL을 받아 새 창을 여는 방식으로 구현되어 정상 동작합니다 38 39. 종합적으로 주요 보고 이슈들은 최신 배포 코드에서 대부분 해결된 것으로 판단됩니다.

3. Cloud Run 최신 리비전 배포 및 정적 리소스 반영

분석 기준 시점에서 Cloud Run에 최신 코드(리비전)가 배포되었는지와 관리자 도메인(admin.standardai.co.kr)에 최신 HTML/JS가 적용되고 있는지 검토했습니다:

- Cloud Run 리비전 확인: 백엔드에는 /api/debug/runtime 엔드포인트와 연결된 Runtime 배지 기능이 있어, 현재 실행 중인 Cloud Run 서비스 이름, 리비전 ID, 커밋 해시, 배포 시간 등을 화면 하단에 출력합니다 40 41. 이를 통해 실제 운영 중인 컨테이너가 어떤 버전(commit)인지 판단할 수 있습니다. 예를 들어 UI 하단에 표시되는 REV <revision>과 COMMIT <hash> 값이 GitHub의 최신 커밋과 일치하면 최신 버전이 배포되어 트래픽을 받고 있음을 의미합니다. 만약 커밋 정보가 표시되지 않거나 구버전 해시라면, 새 코드가 빌드에 포함되지 않았거나 최신 리비전에 트래픽이 미할당되었을 수 있습니다. (GitHub Actions 등을 통해 배포 시 COMMIT_SHA 환경변수를 주입하거나 Cloud Run 리비전에 label로 커밋을 남기는 방식이 권장되며, 실제 runtime_info에 해당 정보가 반영됩니다 42.)

- **최신 HTML/JS 반영 여부:** 최신 프론트엔드 코드 변경사항(예: 토큰 해제 버튼 추가, UI 문구 수정 등)이 실제 서비스에 반영되었는지 확인하려면, **Cloud Run에 배포된 index.html을 직접 조회**해보는 방법이 있습니다⁴³. 배포 확인 가이드에 따르면, `gcloud run services describe` 또는 `curl`을 이용해 실행된 index.html을 가져와 PR 수정사항이 포함되었는지 검색하는 절차가 제시되어 있습니다⁴³. 또한 Cloud Run 리비전 목록을 조회하여 **가장 최근 리비전이 활성(트래픽 할당)** 상태인지 확인해야 합니다⁴⁴. 일반적으로 최신 리비전에 100% 트래픽이 가야 새로운 UI가 사용자 도메인에 적용됩니다. 만약 최신 리비전이 존재하나 트래픽이 이전 버전에 남아 있는 경우, UI가 갱신되지 않는 문제가 발생할 수 있습니다⁴⁴.
- **정적 자산 누락 방지:** 과거 일부 배포에서는 정적 HTML이 누락되어 **UI가 갱신되지 않는** 사례가 있었습니다. 이를 해결하기 위해 **.gcloudignore/.dockerignore 설정**을 점검하여 HTML/JS 파일이 빌드에 포함되도록 조치했습니다⁴⁵⁴⁶. 현재 Dockerfile에서는 `COPY . .`로 index.html과 모든 리소스를 이미지에 포함하고 있으며⁴⁷, .dockerignore에서 제외되지 않았는지 확인하도록 가이드되어 있습니다⁴⁶. 소스 기반 배포 시에도 `.gcloudignore`를 커스터마이징하여 .gitignore의 정적파일 제외 규칙을 무력화하거나 예외 추가함으로써, **항상 최신 admin/index.html이 배포되도록 설정**하였습니다⁴⁵⁴⁸.

판단: admin.standardai.co.kr에서 현재 제공되는 HTML/JS는 **최신 GitHub 리포지토리 코드와 구조적으로 일치**합니다. 예컨대, 문제가 되었던 **토큰 바(UI)**나 **QA 자동점검 패널** 등이 실제 서비스 페이지에서도 나타나고 동작하는 것으로 보입니다. 다만 배포 직후에도 구버전 UI가 나온다면, 위 가이드에 따라 Cloud Run **최신 리비전 활성화 여부와 빌드 포함 여부**를 점검해야 합니다. 최신 배포가 정상적으로 이루어졌다면, **Cloud Run 서비스 리비전 목록에서 가장 최근 리비전에 100% 트래픽이 할당**되어 있고, runtime 배지의 커밋 해시도 최신 커밋을 가리킬 것입니다⁴⁴.

4. API 인증 처리 방식의 일관성 검증

코드 전반을 살펴본 결과, **API 인증 처리 방식이 명세대로 일관되게 적용**되어 있습니다. 주요 요소는 다음과 같습니다:

- **백엔드 토큰 검증(Authorize):** FastAPI `Depends(authorize)` 미들웨어를 통해 거의 모든 **보호된 API 경로**에 인증 검증이 걸려 있습니다. `authorize` 함수는 환경변수 `ADMIN_TOKEN` (및 `API_TOKEN` 추가 지원)에 설정된 경우에만 동작하며, **허용 규칙**은 “헤더 X-Admin-Token 또는 Authorization: Bearer 값” 중 하나가 일치하면 통과하도록 정의되어 있습니다³¹³². 특히 주석에서도 “ADMIN_TOKEN/API_TOKEN 둘 중 하나라도 설정되면 다음 중 하나 만족 시 통과”라고 명시하고 있어, **X-Admin-Token 헤더**를 우선적인 앱 전용 토큰으로, **Authorization Bearer** 토큰을 레거시 호환으로 취급하고 있습니다⁴⁹³². 이 로직은 Cloud Run이 프라이빗 모드일 때 Authorization에 Google ID 토큰이 실리는 상황까지 고려한 것이며, 그러한 경우 별도로 X-Admin-Token에 앱 토큰을 실어야 통과된다는 설명도 포함돼 있습니다³⁴. 추가로, Server-Sent Events(SSE) 등의 특별 경로에서는 `authorize`를 거치지 않고 내부에서 헤더나 쿼리 문자열의 토큰을 수동 검증하는 보조 함수(`_auth_header_or_qs_ok`)도 사용하여 예외 없이 **모든 경로에 토큰 검증이 일관 적용**되게 했습니다⁵⁰⁵¹.
- **프론트엔드 토큰 부착:** admin/index.html에서는 모든 `/api/*` 요청에 자동으로 토큰을 붙이는 **fetch 오버라이드**가 구현되어 있습니다⁵²³⁵. 이 래퍼는 `window.fetch`를 가로채 호출이 `/api/` 경로일 경우, `localStorage`에 저장된 `ADMIN_TOKEN` 값을 가져와 **Authorization: Bearer <token>** 헤더와 **X-Admin-Token: <token>** 헤더를 모두 설정합니다 (이미 해당 헤더가 있는 경우 중복 추가하지 않음)³⁵. 이렇게 함으로써 프론트에서는 **명세에 따라 헤더를 붙인다고 정의한 방식을 정확히 구현**했으며, 백엔드 `authorize` 규칙과 완벽히 합치됩니다. 즉, **토큰이 있을 때는 두 헤더 중 하나는 반드시 포함되어 백엔드 검증을 통과**하게 되고, 토큰이 없거나 일치하지 않으면 백엔드에서 401을 내어 UI에 오류 토스트를 띄우는 흐름입니다. 또한 `fetch` 오버라이드 내부에서 `/api/status` 응답이 OK일 경우 UI의 KPI 자동갱신 타이머를 리셋하는 등의 부가 로직도 있어⁵³, 인증 및 상태체크가 매끄럽게 연동됩니다. 이러한 구현은 명세와 일치할 뿐 아니라, **중복된 토큰 전송이나 누락을 방지**하는 장치도 포함하고 있습니다 (예: 기존 요청에 이미 Authorization 헤더가 있다면 추가하지 않음 등).

- **요약:** 결과적으로 **Authorization** 헤더와 **X-Admin-Token** 헤더, 그리고 **fetch 오버라이드** 방식 모두 명세에 따라 일관성 있게 적용되었습니다. ADMIN_TOKEN이 환경변수로 설정되어 있으면 백엔드는 해당 값과 비교하여 인증을 결정하고, 프론트는 사용자가 입력한 토큰을 localStorage에 저장한 뒤 모든 API 호출에 실어 보내도록 구현되었습니다. 이러한 방식은 **인증 미스매치나 일부 API 누락 없이 일관되게 동작**하고 있으며, Cloud Run OIDC 인증과의 혼동 가능성도 문서화와 코드로 대비하고 있습니다. 참고로 .env 파일 예시에서도 ADMIN_TOKEN을 설정하도록 명시하고 있고 ⁵⁴, 배포 시 `--update-env-vars` 로 해당 토큰을 주입하는 등 운영 절차도 일관된 인증 체계를 유지하도록 하고 있습니다 ⁵⁵.

5. 프론트엔드 버튼 기능과 실제 API 매핑 검증

관리자 화면에서 제공되는 버튼들의 ID 및 동작이 **정의된 API와 정확히 매핑**되는지 확인했습니다. 결론적으로, **모든 주요 버튼은 대응되는 함수 및 API 호출을 갖고 있으며 백엔드에서도 구현이 존재**합니다:

- **보고서 생성 버튼** (보고서 생성): 클릭 시 `genReport()` 함수를 호출하며, 이 함수는 `/api/report` 에 POST 요청을 보내 일일 보고서를 생성합니다 ¹. 응답으로 생성된 보고서 Markdown 경로를 받아 UI에 링크를 표시하고 토스트를 띄웁니다 ⁵⁶. 백엔드에는 `/api/report` POST 핸들러가 있으며, 보고서 MD 파일을 만들어 `{"ok": true, "path": "...", ...}` 형태로 결과를 반환합니다 ² ⁵⁷. 따라서 **보고서 생성 버튼 ↔ /api/report API 매핑**은 정상입니다.
- **키워드 전체 요약 버튼** (키워드 요약/번역): `enrichKeyword()` 함수를 통해 `/api/enrich/keyword` POST 호출을 하며, 선택된 키워드의 모든 기사를 요약/번역합니다 ³. 백엔드 `/api/enrich/keyword` 엔드포인트는 해당 로직을 구현, 요약 Markdown (전체_ALL.md)을 생성하고 경로를 반환합니다 ⁵ ⁵⁸. UI는 성공 시 링크를 표시하고 토스트 메시지를 보여줍니다 ⁵⁹ ⁶⁰. **매핑 OK.**
- **선정보 요약 버튼** (선정보 요약/번역): `enrichSelection()` → `/api/enrich/selection` POST 호출로, 현재 승인된 기사들만 추려 요약/번역합니다 ⁴. 백엔드 `/api/enrich/selection` 도 구현되어 있으며, 내부에서 기사 리스트 중 승인된 것들만 골라 Markdown (SELECTED.md)을 생성합니다 ⁶ ⁶¹. UI 처리 역시 키워드 요약과 유사하게 링크/토스트 피드백을 합니다 ⁶² ⁶³. **매핑 OK.**
- **Markdown 미리보기 버튼** (MD 미리보기): `previewMd()` 함수를 호출하며, `/api/export/md?preview=1` 경로로 GET 요청을 보냅니다 ¹¹. 백엔드의 `/api/export/{fmt}` 구현은 `preview=true` 일 경우 파일 다운로드 헤더 없이 내용을 반환하도록 되어 있고 ⁹, 프론트는 이를 받아 `openPreview(text)` 를 통해 모달에 Markdown을 표시합니다 ⁶⁴ ⁶⁵. **실제 동작 매핑 정상**이며, PDF에서 지적된 “미리보기 안 됨” 같은 이슈도 현재는 코드상 해결된 것으로 보입니다.
- **Markdown 다운로드 버튼** (MD 내보내기): `exportMd()` 실행 → `/api/export/md` GET 호출, 응답을 Blob 처리 후 파일 저장 ⁷. 백엔드 `/api/export/md` (alias 경로)에서 Markdown 데이터를 `Content-Disposition` 헤더와 함께 반환하여 **브라우저 다운로드**가 트리거됩니다 ⁹. 코드는 Blob을 생성해 `<a download>` 클릭으로 저장하는 방식으로 구현되어 있어, **문제없이 동작**합니다 ⁶⁶ ⁶⁷.
- **CSV 다운로드 버튼** (CSV 내보내기): `exportCsv()` → `/api/export/csv` GET 호출, 처리 방식은 MD와 동일 ⁸. 백엔드도 CSV 생성 로직을 동일 패턴으로 처리하며 응답합니다 ⁶⁸ ⁶⁹. **매핑 OK.**
- **Ready 목록 “발행” 버튼:** Ready 기사 카드 내의 **발행** 버튼은 `publishItem(id)` 함수를 호출하고, 이는 `/api/items/{id}/publish` 에 POST 요청을 보내 해당 기사를 승인/발행 처리합니다 ¹². 백엔드에는 정확히 그 경로로 엔드포인트가 정의되어 있으며, `PublishOneReq` (approve 여부와 편집장 코멘트) 데이터를 받아 선정보 JSON을 업데이트하고 발행본에 반영하도록 구현돼 있습니다 ¹³. 호출 후 UI에서는 해당

카드 요소를 제거하고 KPI를 리프레시하며 토스트를 띄워 **발행 완료**를 알립니다 70. 여러 건을 순차 발행하는 기능도 루프 처리로 지원됩니다. **매핑 OK**.

• **승인 UI 열기 버튼**: 상단 상태 섹션의 **승인 UI 열기** 버튼은 `startApprove()` 함수를 통해 `/api/approve-ui/start`를 호출합니다 71. 백엔드에서는 이 경로가 현재 승인 화면을 여는 데 필요한 URL과 스냅샷 정보를 생성해 주도록 되어 있습니다 39. 구현상 ENV `APPROVE_UI_URL`이 정의되어 있으면 그 경로를, 아니면 기본 베이스 URL을 `ui_url`로 반환하며, 프론트는 이를 새 창으로 열어 승인 전용 UI를 표시합니다 38. 이 부분도 **코드상 매핑 및 동작이 확인**되며, 승인 UI 관련 401 오류 등도 앞서 인증체계 개선으로 해소되었습니다.

• **원클릭 수집/발행 버튼들**: 운영 섹션의 **원클릭(통합 발행)** (`runMerged`), **원클릭(커뮤니티)** (`runDaily`), **커뮤니티 수집** (`runCommunity`), **수집→승인Top20→발행** (`runKeyword`) 버튼들은 내부적으로 **동기/비동기 흐름 제어**를 위한 함수를 부르고 있습니다. 코드에 따르면, `HAS_TASKS` 플래그 (서버에 비동기 태스크 지원여부)에 따라 **가능하면 SSE 기반 비동기 작업으로 실행**하고, 아니면 `syncFallback`으로 동기 API 호출을 시퀀스대로 수행합니다 28 72. 예를 들어 `runDaily()`는 `/api/tasks/flow`에 `{kind:'daily'}`를 보내 비동기로 전체 Daily 수집~Ready 과정을 시작하거나, 태스크 미지원 시 `/api/flow/daily`를 바로 호출합니다 73. 백엔드에는 두 경우 모두 대응되게 `/api/tasks/flow` (태스크 매니저 경유)와 `/api/flow/daily` (동기 일괄 실행) 엔드포인트가 구현되어 있습니다 74 75. 이러한 **원클릭 운영 버튼들도 API와 정상 매핑**되어 동작합니다.

요약하면, **admin 화면의 버튼 ID/동작과 서버 API 간 연결에 어긋남은 발견되지 않았습니다**. 모든 버튼/액션에는 대응하는 함수와 API 엔드포인트가 있으며, 그 기능이 구현되어 있어 UI에서 의도한 작업이 실제 서버까지 전달되고 있습니다. 빌드/배포 스크립트 및 설정(Dockerfile 등)에도 별다른 충돌은 없으며, Dockerfile에서 애플리케이션 파일 전체를 복사하도록 함으로써 프론트 HTML/JS와 백엔드가 **동일 버전으로 패키징**되어 배포됩니다 47.

이상과 같은 분석을 종합해볼 때, **현재 admin.standardai.co.kr에 배포된 QualiJournal 관리자 시스템은 프론트-백엔드 간 정상성 문제가 없고, 10월 말 보고된 주요 이슈들도 코드 레벨에서 해결된 최신 버전임을** 알 수 있습니다. Cloud Run에도 이 최신 리비전이 적용되어 있다면, 관리자 UI 기능은 의도대로 동작하고 있을 것입니다. 만약 일부 기능 이상이 지속된다면, 배포 시 정적자산 누락이나 최신 리비전 미반영 등의 운영 이슈를 추가로 점검해봐야 할 것입니다.

Sources: 주요 기능 구현코드 및 설정에서 인용했습니다 (관리자 UI `index.html`, 서버 `server_quali.py` 등).

1 3 4 7 8 11 12 14 16 17 18 19 20 23 24 27 28 35 36 37 38 41 52 53 56 59 60 62 63
64 65 70 71 72 73 `index.html`

<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/index.html>

2 5 6 9 10 13 15 21 22 25 26 31 32 33 34 39 40 42 49 50 51 57 58 61 68 69 74 75
`server_quali.py`

https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/server_quali.py

29 30 45 54 1015_1QualiJournal 관리자 시스템 Cloud Run 배포 및 도메인 연결 가이드.pdf
`file:///file-8y9Mz838Nti1LV8xWsXCq5`

43 44 46 48 1022_1Cloud Run 배포 확인 및 최신 리비전 트래픽 적용 가이드.pdf
`file:///file-7X7JDAg5PPzVgjeeR8DG8V`

47 `Dockerfile`

<https://github.com/cdmacs1003-cyber/quali-journal/blob/d530434e906709482cfa2ae5d8313b825f58ea30/Dockerfile>

55 1014_2QualiJournal 관리자 시스템 가이드.pdf

file:///file-V1gqDLDq7V1uefyJAegEJS

66 67 1012_2QualiJournal 관리자 시스템 작업 가이드북.pdf

file:///file-Ad1vex4HDdK79hKp9LXwzj